```
[ ]: lambda function
     a mabda function is a small,anonymous (nameless) function in python.
         .it defined using keyword lamdba insteaad of def.
         .it can take any number of arguments but must contain only one expression.
         .expression is automatically returned(no need to use return)

     syntax
     lambda arguments:expression


     why to use lambda?
     .useful for short,simple operation
     .saves code(one-liner)
     .often used with map(),filter(),reduce() or as inline function


     when not use lambda?
     .for complex logic(use def instead)
     .when readability is more important than shortness
```

```
[15]: s = lambda a,b : a+b
```

```
[16]: s(3,4)
```

```
[16]: 7
```

```
[18]: ev = lambda n : "even" if (n%2 == 0) else "odd"
```

```
[19]: ev(4)
```

```
[19]: 'even'
```

```
[20]: ev(33)
```

```
[20]: 'odd'
```

```
[ ]: Data structures are ways of organizing and storing data
     so that they can accessed and modified effeciently.
     *built in data structures
     *user defined dsa

     python provides both built in ds and allows us to implement user defined ds

     1.built in ds in python
     these come pre-implemented with python
     list,tuple,set,dict
```

```
[ ]: list
     it is a multivalue datatype which stores multiple data in one variable of all different datatypes datas
     list is represented by : [] ,list()

     properties:
     .its heterogenous in nature
     .it allow duplicates
     .its mutable in nature
     .its ordered in nature
```

```
[22]: ls = [23,2,3,8+2j,"jj",True,True,"jj"]
```

```
[23]: ls
```

```
[23]: [23, 2, 3, 2j, 'jj', True, True, 'jj']
```

```
[27]: ls.append("jk")
```

```
[28]: ls
```

```
[28]: [23, 2, 3, 2j, 'jj', True, True, 'jj', 'jungkook', 'jk']
```

```
[30]: ls.insert(2,100)
      ls

[30]: [23, 2, 100, 3, 2j, 'jj', True, True, 'jj', 'jungkook', 'jk']

[32]: ls[9] = "jeonghan"
      ls

[32]: [23, 2, 100, 3, 2j, 'jj', True, True, 'jj', 'jeonghan', 'jk']

[33]: ls.pop(5)

[33]: 'jj'

[34]: ls.pop()

[34]: 'jk'

[35]: ls

[35]: [23, 2, 100, 3, 2j, True, True, 'jj', 'jeonghan']

[36]: ls.append("jk")

[37]: ls

[37]: [23, 2, 100, 3, 2j, True, True, 'jj', 'jeonghan', 'jk']

[38]: ls.remove(True)

[39]: ls

[39]: [23, 2, 100, 3, 2j, True, 'jj', 'jeonghan', 'jk']

[40]: ls.clear()
      ls

[40]: []
```

```
[44]: del ls
      ls

      ---------------------------------------------------------------------------
      NameError                                 Traceback (most recent call last)
      Cell In[44], line 1
      ----> 1 del ls
            2 ls

      NameError: name 'ls' is not defined

[45]: ev = []
      od = []
      for i in range(1,11):
          if i %2 == 0:
              ev.append(i)
          else:
              od.append(i)
      ev

[45]: [2, 4, 6, 8, 10]

[46]: od

[46]: [1, 3, 5, 7, 9]

[47]: print(ev,od)

      [2, 4, 6, 8, 10] [1, 3, 5, 7, 9]

[49]: sl = [2,6,3,8,9,1]
      s = []
      for i in sl:
          s.append(i*i)

      print(s)

      [4, 36, 9, 64, 81, 1]
```

[2]:
```
l1 = [2,4,1,5,2,4,9,8,5]
#remove duplicates
l2=[]
for i in l1:
    if i not in l2:
        l2.append(i)
print(l2)
```

```
[2, 4, 1, 5, 9, 8]
```

[3]:
```
l1 = [1,2,3,4,5]
l2=[4,5,6,7,8]
l3 = l1+l2
print(l3)
```

```
[1, 2, 3, 4, 5, 4, 5, 6, 7, 8]
```

[19]:
```
l1 =[1,2,3,4,5]
l2=[4,5,6,7,8]
l3=[]
for i in range(0,len(l1),1):
    l3.append(l1[i] + l2[i])
print(l3)
```

```
[5, 7, 9, 11, 13]
```

[ ]:
```
tuple
it is a multivalue datatype which stores multiple data in one variable of all different datatypes datas
tuple is represented by : () ,tuple()

properties:
.its heterogenous in nature
.it allow duplicates
.its immutable in nature
.its ordered in nature
```

[20]:
```
t1 = (23,2,3,8+2j,"jj",True,True,"jj")
```

[21]:
```
t1
```

```
(23, 2, 3, (8+2j), 'jj', True, True, 'jj')
```

[21]:
```
t1
```

```
(23, 2, 3, (8+2j), 'jj', True, True, 'jj')
```

[22]:
```
t1.append(45)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[22], line 1
----> 1 t1.append(45)

AttributeError: 'tuple' object has no attribute 'append'
```

[23]:
```
t1.insert(1,90)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[23], line 1
----> 1 t1.insert(1,90)

AttributeError: 'tuple' object has no attribute 'insert'
```

t1.add(45)

[29]:
```
t1 = (23,2,3,"python",8+2j,"jj",True,True,"jj")

l1 = list(t1)
l1[3]="java"

t1=tuple(l1)
t1
```

```
(23, 2, 3, 'java', (8+2j), 'jj', True, True, 'jj')
```

[30]:
```
l1 = (1,2,3,4,5)
l2=(4,5,6,7,8)
l1+l2
```

```
(1, 2, 3, 4, 5, 4, 5, 6, 7, 8)
```

[35]:
```
#list

t1 = [1,2,3,4,5]
t2=[4,5,6,7,8]
t3=()
for i in range(0,len(t1)):
    t3 = t3+((t1[i],t2[i]),)
t3
```

[35]: ((1, 4), (2, 5), (3, 6), (4, 7), (5, 8))

[36]:
```
#tuple

t1 = (1,2,3,4,5)
t2=(4,5,6,7,8)
t3=()
for i in range(0,len(t1)):
    t3 = t3+((t1[i],t2[i]),)
t3
```

[36]: ((1, 4), (2, 5), (3, 6), (4, 7), (5, 8))

[ ]:
```
set
it is a multivalue datatype which stores multiple data in one variable of all different datatypes datas
tuple is represented by : {},set()

properties:
.its heterogenous in nature
.it wont allow duplicates
.its mutable in nature
.its unordered in nature
```

[37]:
```
s1 = {23,2,3,"python",8+2j,"jj",True,True,"jj"}
s1
```

[37]: {(8+2j), 2, 23, 3, True, 'jj', 'python'}

[38]:
```
s1.add("jk")
s1
```

[38]: {(8+2j), 2, 23, 3, True, 'jj', 'jk', 'python'}

[39]:
```
s1.add(2000)
s1
```

[39]: {(8+2j), 2, 2000, 23, 3, True, 'jj', 'jk', 'python'}

[40]:
```
s1.remove(2)
s1
```

[40]: {(8+2j), 2000, 23, 3, True, 'jj', 'jk', 'python'}

[41]:
```
s1.add(4000)
s1
```

[41]: {(8+2j), 2000, 23, 3, 4000, True, 'jj', 'jk', 'python'}

[ ]:
```
s1={1,2,3,4,5}
s2={4,5,6,7,8}
o1 = {1,2,3,4,5,6,7,8}
o2 = {1,2,3}
o3 = {6,7,8}
o4 = {4,5}
o5 = {6,7,8}
```

[47]:
```
s1.union(s2)
```

[47]: {1, 2, 3, 4, 5, 6, 7, 8}

[48]:
```
s1.difference(s2)
```

[48]: {1, 2, 3}

[49]:
```
s2.difference(s1)
```

[49]: {6, 7, 8}

[50]:
```
s1.intersection(s2)
```

[50]: {4, 5}

[55]:
```
s1.symmetric_difference(s2)
```

[55]: {1, 2, 3, 6, 7, 8}